

UNIVERSIDAD INDUSTRIAL DE SANTANDER

Facultad de Ingenierías Físico-Mecánicas

INFORME DE PROYECTO INTEGRADOR

*Sistema para la infraestructura y modelo predictivo de puntos de carga
(Electrolineras) para vehículos eléctricos en el área metropolitana de Bucaramanga*

Asignaturas:

Matemáticas Discretas — Algoritmos y Programación

Profesores:

Cesar Augusto Aceros Moreno, PhD

Rosaura Gutiérrez Almeyda, MSc

Integrantes del grupo:

Daniel Rodriguez — Código: 2252905

Maria Jose Niño — Código: 2252913

Lauren Figueredo — Código: 2252900

Mayo de 2026

1. Introducción

El crecimiento sostenido de los vehículos eléctricos en Colombia y en particular en el área metropolitana de Bucaramanga plantea un reto concreto para la planificación urbana: garantizar que la infraestructura de recarga eléctrica sea suficiente, bien distribuida y capaz de anticipar la demanda futura. En la actualidad, muchas ciudades cuentan con estaciones de carga escasas o mal ubicadas, lo que genera lo que en la literatura se denomina «ansiedad de autonomía» en los conductores y frena la adopción masiva del transporte eléctrico.

Este proyecto aborda esa problemática desde el campo de las Matemáticas Discretas y la programación en Python, integrando teoría de grafos, algoritmos clásicos de búsqueda de caminos mínimos y modelos de aprendizaje automático supervisado. El resultado es un sistema computacional que, a partir de la red vial real del área metropolitana obtenida de OpenStreetMap, simula recorridos de vehículos eléctricos, detecta los momentos en que la batería cae por debajo del umbral crítico y calcula la ruta más corta hacia la estación de carga más próxima. Con la información acumulada de esas simulaciones se entrena un modelo de bosque aleatorio capaz de predecir si un corredor determinado requerirá recarga, y se identifican las zonas de mayor demanda para proponer la ubicación óptima de nuevas electrolineras.

El alcance del trabajo cubre el área metropolitana de Bucaramanga —Bucaramanga, Floridablanca, Girón y Piedecuesta— considerando ocho electrolineras existentes y diez puntos fijos de referencia que incluyen sedes universitarias, centros de convenciones y zonas recreativas. La selección de dos vehículos representativos —uno de alta gama y uno de baja gama— permite comparar el comportamiento de la demanda en escenarios de diferente autonomía y consumo energético.

2. Modelaje y Especificación del Problema

2.1 Representación como grafo ponderado

La red vial del área metropolitana se modela como un grafo dirigido ponderado $G = (V, E, w)$, donde cada vértice $v \in V$ corresponde a una intersección o punto relevante de la malla vial, cada arista $(u, v) \in E$ representa un segmento vial con sentido permitido de circulación, y la función de peso $w: E \rightarrow \mathbb{R}^+$ asigna a cada arista la longitud en metros del segmento correspondiente, obtenida directamente de OpenStreetMap a través de la biblioteca OSMnx.

Dentro del conjunto V se identifican dos subconjuntos de especial interés:

- $P \subset V$: conjunto de diez puntos fijos de referencia (universidades, CENFER, PTAR, etc.) que actúan como posibles orígenes y destinos de los recorridos simulados.
- $E_p \subset V$: conjunto de ocho electrolineras existentes, representadas como nodos snap al grafo mediante el nodo de la red más cercano a sus coordenadas geográficas reales.

El grafo se construye componiendo los subgrafos descargados para cada uno de los cuatro municipios del área metropolitana mediante la operación de composición de grafos de NetworkX, garantizando la conectividad entre los municipios limítrofes.

2.2 Definición formal del problema del camino más corto

Dado el grafo G y dos nodos $s, t \in V$, el camino más corto $d(s, t)$ se define como:

$$d(s, t) = \min \{ \sum w(e_i) : (e_1, e_2, \dots, e_k) \text{ es un camino de } s \text{ a } t \text{ en } G \}$$

El sistema resuelve dos variantes de este problema en tiempo de ejecución:

1. Camino entre puntos de referencia: para cada recorrido simulado, se calcula $d(p_i, p_j)$ con $p_i, p_j \in P$ usando el algoritmo de Dijkstra implementado en NetworkX (nx.shortest_path con weight='length').
2. Camino al punto de carga más cercano: cuando la batería del vehículo cae al umbral crítico en el destino p_j , se calcula $\min_{e \in E_p} d(p_j, e)$ para identificar la electrolinera más próxima.

2.3 Variables y parámetros del problema

Variable / Parámetro	Descripción	Valor / Fuente
Autonomía máxima (km)	Distancia máxima con batería llena	175 km (Citroën e-C3) / 395 km (Mercedes G580)
Batería total (kWh)	Capacidad energética del vehículo	29.8 kWh / 116 kWh
Umbral de recarga (%)	Nivel mínimo de batería antes de buscar electrolinera	$\leq 20 \%$
Nivel de recarga (%)	Nivel al que se recarga la batería al usar la electrolinera	80 %
$ P = 10$	Puntos fijos de referencia	Sedes UIS, UNAB, UTS, UPB, CENFER, PTAR, Catay
$ E_p = 8$	Electrolineras existentes	Homecenter, CC Cacique, CC Canaveral, Terpel, etc.

2.4 Vehículos eléctricos seleccionados

La selección de los vehículos se realizó desde la base de datos pública EV Database (ev-database.org). Se eligieron un representante de baja gama y uno de alta gama para evidenciar el contraste en autonomía, consumo y frecuencia de recarga:

Modelo	Autonomía (km)	Batería (kWh)	Gama
Citroën ë-C3	175	29.8	Baja
Mercedes-Benz G 580 EQ	395	116	Alta

El consumo energético por kilómetro se calcula dividiendo la capacidad total de la batería entre la autonomía declarada. Para el Citroën e-C3 esto equivale a 0.170 kWh/km, mientras que el

Mercedes G580 consume 0.294 kWh/km. Esta diferencia influye directamente en la frecuencia de recarga observada durante las simulaciones.

3. Descripción del Algoritmo

3.1 Arquitectura general del sistema

El sistema se organiza como un menú interactivo de siete opciones que siguen un flujo secuencial de dependencias. La figura conceptual a continuación resume la arquitectura funcional:

Opción 0 → Genera el archivo JSON con coordenadas

Opción 1 → Inicializa el grafo y carga datos

Opción 2 → Simula n recorridos y registra eventos de recarga

Opción 3 → Construye el dataset tabular de recorridos

Opción 4 → Entrena el modelo de Random Forest

Opción 5 → Visualiza el mapa de rutas

Opción 6 → Analiza y recomienda nuevas electrolineras

3.2 Inicialización del grafo (Opción 1)

Al inicializar, el sistema descarga los grafos viales de los cuatro municipios del área metropolitana usando OSMnx y los compone en un único grafo mediante `nx.compose_all()`. Luego identifica el nodo más cercano en la red vial para cada punto de referencia y cada electrolinera usando `ox.distance.nearest_nodes()`, resolviendo el problema de snap de coordenadas GPS a nodos de la red.

El sistema mantiene dos diccionarios de caché (`_cache_dist_ruta` y `_cache_dist_electrolinera`) para evitar recalcular caminos ya computados, lo que reduce significativamente el tiempo de simulación cuando se realizan múltiples recorridos entre los mismos pares de nodos.

3.3 Simulación de recorridos (Opción 2)

El núcleo del sistema. Para cada uno de los n recorridos solicitados por el usuario:

3. Se seleccionan aleatoriamente un origen y un destino distintos del conjunto P usando `numpy.random.choice` con `replace=False`.
4. Se calcula el camino más corto entre origen y destino usando `nx.shortest_path` con el peso 'length', implementación del algoritmo de Dijkstra.
5. Se recorre el camino arista por arista, descontando de la batería actual el consumo proporcional a la longitud de cada segmento.
6. Al llegar al destino, si la batería ha caído al 20% o menos, se activa el protocolo de recarga: se busca la electrolinera más cercana, se registra el uso en los contadores de frecuencia y se restaura la batería al 80%.

7. Se almacena un registro completo del recorrido en la lista `registros_recorridos`.

La validación de entrada en esta opción cubre: campo vacío, valores negativos, caracteres no numéricos y valores fuera del rango permitido (1 a 10.000 recorridos).

3.4 Generación de dataset y entrenamiento IA (Opciones 3 y 4)

Los registros acumulados durante las simulaciones se consolidan en un DataFrame de pandas que se exporta como archivo CSV. Cada fila representa un recorrido y contiene las siguientes características relevantes para el modelo supervisado:

- `distancia_ruta_km`: longitud total del camino en la red vial.
- `consumo_estimado_kwh`: energía consumida estimada según el modelo de vehículo.
- `dist_electrolinera_min_km`: distancia al punto de carga más cercano desde el destino.
- `necesita_electrolinera`: etiqueta binaria (0/1) que indica si se requirió recarga.

Con estas características se entrena un RandomForestClassifier de scikit-learn con 100 árboles. El conjunto de datos se divide en 80% para entrenamiento y 20% para prueba. El sistema reporta la precisión (accuracy), el tiempo de entrenamiento y el tiempo de predicción, junto con el reporte de clasificación detallado que incluye precisión, sensibilidad y puntuación F1 para cada clase.

3.5 Análisis de nuevas electrolineras (Opción 6)

Con base en los contadores de frecuencia acumulados durante las simulaciones, el sistema realiza tres análisis:

8. Ranking de electrolineras existentes por número de recargas atendidas, calculando el porcentaje de participación de cada una.
9. Análisis por tipo de vehículo, mostrando qué estaciones son más demandadas según el vehículo simulado.
10. Recomendación de zonas para nuevas electrolineras: identifica los cinco corredores con mayor número de eventos de recarga y calcula el punto medio geográfico de cada uno como ubicación sugerida para una nueva estación. Las electrolineras que superan el 30% de las recargas totales se marcan como saturadas.

El análisis completo se exporta como archivo JSON para uso posterior por parte de tomadores de decisiones.

4. Descripción de la Implementación en Python

4.1 Estructura modular del código

El programa se desarrolló en Python sobre Google Colab, organizado en funciones independientes para cada opción del menú. Esta estructura modular sigue el principio de

responsabilidad única: cada función encapsula una funcionalidad completa y puede invocarse de forma independiente. El flujo de control principal es un ciclo `while True` controlado por centinela (opción 7 = salir) que valida la entrada del usuario antes de despachar la acción correspondiente.

4.2 Estructuras de datos utilizadas

Estructura	Uso en el sistema	Ejemplo en el código
Lista (array 1D)	Frecuencia de recargas por electrolinera, lista de nombres	<code>frecuencia_electrolineras = [0] * n</code>
Diccionario (mapa)	Caché de distancias, frecuencia por corredor, por vehículo	<code>_cache_dist_ruta = {}</code>
Lista de diccionarios	Registros de recorridos con todos sus atributos	<code>registros_recorridos.append({...})</code>
DataFrame (pandas)	Dataset tabular para entrenamiento del modelo IA	<code>pd.DataFrame(registros_recorridos)</code>
Grafo NetworkX	Representación de la red vial completa del AMB	<code>nx.compose_all(grafos_municipios)</code>
Archivo JSON	Almacenamiento de coordenadas y resultados de análisis	<code>json.dump(datos, f, ensure_ascii=False)</code>
Archivo CSV	Exportación del dataset de recorridos simulados	<code>dataset_global.to_csv('dataset_recorridos.csv')</code>

4.3 Validación de entradas

El sistema valida de forma exhaustiva todas las entradas del usuario. Para el menú principal y la selección de vehículo se comprueban: campo vacío, caracteres no numéricos, valores fuera del rango de opciones disponibles. Para la cantidad de recorridos se comprueban adicionalmente: valores negativos (verificando si la cadena inicia con '-') y valores mayores a 10.000.

Este enfoque de validación capa por capa garantiza que el programa no colapse ante entradas inesperadas y ofrece mensajes de error descriptivos que orientan al usuario hacia la corrección.

4.4 Gestión de archivos

El sistema trabaja con tres formatos de archivo distintos según lo requerido en el proyecto:

- JSON (.json): para la carga de datos de entrada (coordenadas de puntos y electrolineras) y para la exportación del análisis de nuevas electrolineras.
- CSV (.csv): para la exportación del dataset de recorridos, compatible con herramientas de análisis externas.
- Integración con Google Drive: en el entorno Colab, los archivos generados se descargan automáticamente al equipo del usuario mediante la API `files.download()` de `google.colab`.

4.5 Bibliotecas y herramientas

La implementación hace uso del siguiente ecosistema de bibliotecas Python:

Biblioteca	Función en el proyecto
osmnx	Descarga de grafos viales desde OpenStreetMap y snap de coordenadas
networkx	Construcción, composición y análisis del grafo; cálculo de caminos mínimos (Dijkstra)
numpy	Selección aleatoria de orígenes y destinos con distribución uniforme
pandas	Construcción y manipulación del dataset tabular de recorridos
scikit-learn	Partición train/test, entrenamiento del RandomForestClassifier, métricas de evaluación
matplotlib	Visualización estática del grafo y las rutas simuladas
folium	Generación de mapas interactivos HTML con capas de rutas y puntos
json / csv	Lectura y escritura de archivos de datos en formatos estándar

5. Conexión con Conceptos de Matemáticas Discretas

5.1 Grafos ponderados y teoría de grafos

El núcleo matemático del proyecto es la representación de la red vial como un grafo ponderado dirigido. Los conceptos de nodo (intersección vial), arista (segmento de vía) y peso (longitud en metros) son la abstracción directa de la teoría de grafos aplicada a un problema real. La propiedad de conexión del grafo —verificada mediante `nx.has_path()` antes de calcular rutas— es fundamental para garantizar que exista al menos un camino entre cualquier par de puntos de interés.

5.2 Algoritmo de Dijkstra

El algoritmo de Dijkstra resuelve el problema del camino mínimo en grafos con pesos no negativos, condición que se cumple aquí ya que todas las longitudes de segmentos viales son positivas. La implementación de NetworkX utiliza una cola de prioridad (min-heap) que garantiza una complejidad temporal de $O((V + E) \log V)$, donde V es el número de nodos y E el número de aristas del grafo. Para el área metropolitana de Bucaramanga, el grafo contiene típicamente decenas de miles de nodos e intersecciones, lo que hace que la eficiencia de Dijkstra sea determinante para la viabilidad computacional de la simulación.

5.3 Optimización combinatoria

La búsqueda de la electrolinera más cercana desde un nodo destino es un problema de optimización combinatoria: dado un conjunto E_p de nodos candidatos y un nodo de consulta q , se busca el elemento $e^* \in E_p$ que minimiza $d(q, e)$. El sistema resuelve este problema ejecutando $|E_p| = 8$ consultas de camino mínimo y seleccionando el resultado de menor valor, con los resultados cacheados para evitar recomputo.

5.4 Análisis estadístico y predicción

Las frecuencias de uso de electrolineras y corredores constituyen distribuciones empíricas construidas a partir de las simulaciones. La identificación de electrolineras saturadas (participación > 30%) y la recomendación de zonas candidatas basada en el centroide geográfico de los corredores más demandados son técnicas de análisis de datos que emergen directamente de la observación de estas distribuciones. El modelo de Random Forest añade una capa predictiva que, una vez entrenado, puede generalizar a recorridos no vistos durante la simulación.

6. Resultados Esperados e Impacto

Al ejecutar el sistema con un número representativo de simulaciones (por ejemplo, 500 a 1.000 recorridos por vehículo), se espera obtener:

- Un ranking cuantitativo de las ocho electrolineras existentes, permitiendo identificar cuáles están sobrecargadas y cuáles presentan baja demanda.
- Un mapa de calor implícito de los corredores con mayor demanda de recarga, que orienta la toma de decisiones sobre dónde instalar nuevas estaciones.
- Un modelo predictivo con precisión esperada superior al 85%, capaz de anticipar si un recorrido determinado requerirá recarga antes de que este se complete.
- Coordenadas geográficas sugeridas para la instalación de nuevas electrolineras en zonas de alta demanda no cubiertas por la infraestructura actual.
- Un dataset estructurado y exportable que sirve de insumo para análisis de movilidad más detallados por parte de autoridades municipales o empresas del sector energético.

Desde la perspectiva de movilidad sostenible, el sistema contribuye a reducir la incertidumbre de los conductores de vehículos eléctricos, apoya la planificación de infraestructura basada en datos y alinea la gestión urbana con los objetivos de reducción de emisiones de carbono.

7. Conclusiones

El proyecto demuestra que la teoría de grafos y los algoritmos de caminos mínimos son herramientas matemáticas poderosas para abordar problemas reales de infraestructura urbana. La combinación de OSMnx, NetworkX y scikit-learn permite construir un sistema funcional que va desde la extracción de datos geospaciales hasta la predicción con aprendizaje automático, pasando por la simulación estocástica de comportamientos de vehículos.

La modularidad del código facilita la extensión del sistema: se pueden incorporar nuevos vehículos, ampliar el área geográfica, ajustar los umbrales de recarga o integrar datos reales de demanda. La estructura de validación de entradas garantiza robustez ante el uso interactivo, y el sistema de caché de distancias hace viable la simulación de miles de recorridos en un tiempo razonable.

En términos de Matemáticas Discretas, el proyecto materializa conceptos centrales del curso — grafos ponderados, caminos mínimos, optimización combinatoria— en una aplicación con

impacto social directo, evidenciando la relevancia práctica de las estructuras matemáticas estudiadas durante el semestre.

8. Referencias Bibliográficas

Rosen, K. H. (2011). Discrete Mathematics and its Applications (7.^a ed.). McGraw-Hill Education.

Epp, S. S. (2010). Matemáticas Discretas y sus Aplicaciones (4.^a ed.). Cengage Learning.

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. Computers, Environment and Urban Systems, 65, 126–139.

Hagberg, A., Swart, P., & Chult, D. (2008). Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008), 11–15.

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

EV Database. (2026). Range of electric cars. Recuperado de <https://ev-database.org/cheatsheet/range-electric-car>

OpenStreetMap contributors. (2026). OpenStreetMap. Recuperado de <https://www.openstreetmap.org>

Sipser, M. (2012). Introduction to the Theory of Computation (3.^a ed.). Cengage Learning.